



SIMATS ENGINEERING ASSESSMENT TOOL 1

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS:3
Mode: Take-Home

Total Marks:50
Hours : 1 day

Annexure A

DESIGN TASK

Code of Conduct:

I Murinni Harinath (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Q. No	Understanding of Concepts (25)	Design & Implementation (30)	Parser/Algorithm Construction (20)	Analysis & Validation (15)	Documentation & Presentation (10)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	/ 20	/ 25	/ 20	/ 15	/ 100

54

Design an unambiguous CFG and parse $(id+id)*(id+id)$

$$G \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

Shift-Reduce Parsing Steps

Input: $(id+id)*(id+id)\$$

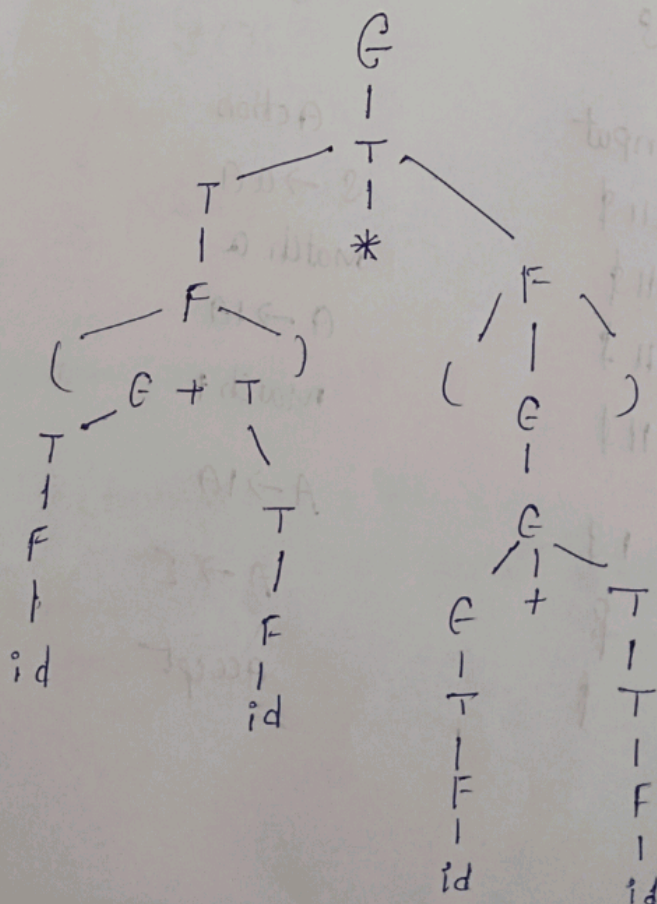
Stack	Input	Action
\$	$(id+id)*(id+id)\$$	shift (
\$(	$id+id)*(id+id)\$$	shift id
\$(id	$+id)*(id+id)\$$	Reduce $F \rightarrow id$
\$(F	$+id)*(id+id)\$$	Reduce $T \rightarrow F$
\$(T	$+id)*(id+id)\$$	Reduce $E \rightarrow T$
\$(E	$+id)*(id+id)\$$	shift +
\$(E+	$id)*(id+id)\$$	shift id
\$(E+id	$)*(id+id)\$$	Reduce $F \rightarrow id$
\$(E+F	$)*(id+id)\$$	Reduce $T \rightarrow F$
\$(E+T	$)*(id+id)\$$	Reduce $E \rightarrow E+T$
\$(E	$)*(id+id)\$$	shift)
\$(E)	$*(id+id)\$$	Reduce $F \rightarrow (E)$
\$(F	$*(id+id)\$$	Reduce $T \rightarrow F$
\$(T	$*(id+id)\$$	Shift *
\$(T*	$(id+id)\$$	Shift (
\$(T*(	$id+id)\$$	Shift id

(+id)* (id+id)

\$ T * (id
\$ T * (F
\$ T * (T
\$ T * (E
\$ T * (E +
\$ T * (E + F
\$ T * (E + T
\$ T * (E .
\$ T * (E)
\$ T * F
\$ T
\$ E

+id)\$
+id)\$
+id)\$
+id)\$
id)\$
)\$
)\$
)\$
)\$
)\$
)\$
)\$
)\$
)\$

Reduce $F \rightarrow id$
Reduce $T \rightarrow F$
Reduce $E \rightarrow T$
Shift +
Shift id
Reduce $T \rightarrow F$
Reduce $E \rightarrow E + T$
Shift)
Reduce $F \rightarrow (E)$
Reduce $T \rightarrow T * F$
Reduce $E \rightarrow T$
Accept



(55)

Design an LL(1) Parser.

$$S \rightarrow aA/b$$

$$A \rightarrow \epsilon A/1$$

First and Follow

- $\text{First}(S) = \{a, b\}$
- $\text{First}(A) = \{\epsilon, 1\}$
- $\text{Follow}(S) = \{\epsilon\}$
- $\text{Follow}(A) = \{\epsilon\}$

LL(1) parsing table

Non-Terminal	a	b	ϵ	\$
S	$S \rightarrow aA$	$S \rightarrow b$	-	-
A	-	-	$A \rightarrow \epsilon A$	$A \rightarrow \epsilon$

initial stack: $\$S$

Stack	input	Action
$\$S$	a \$	$S \rightarrow aA$
$\$Aa$	a \$	match a
$\$A$	\$	$A \rightarrow \epsilon A$
$\$A$	\$	match ϵ
$\$A1$	1 \$	$A \rightarrow \epsilon A$
$\$A$	1 \$	
$\$$	1 \$	$A \rightarrow \epsilon$
		accept

Parse Tree



(58) Recursive Descent Parser

$S \rightarrow aS / \epsilon$

Procedure $S()$

```

{
  if (lookahead == 'a')
  {
    match('a');
    S();
  }
  else
  {
    return;
  }
}

```

Procedure match(x)

```

{
  if (lookahead == x)
    move to next input symbol;
  else
    error;
}

```


Start
 ↓
 Call S()
 ↓
 look ahead = 'a'?
 ↓ yes
 match('a')
 ↓
 Call S()
 ↓
 End.

Demonstration for input: aa

input: aa\$

Step	Action
1	Call S()
2	match first 'a'
3	call S()
4	match second 'a'
5	Call S()
6	look ahead = \$, choose ε
7	Return.

Hence the input aa is accepted by the recursive descent parser.